Assignment 01 - Driver interfaces

How to Write a Device Driver

Mai Thế Tâm
Student ID: SE205068
Course: LOS201
Target: Linux 5.15, ARMv7 Cortex-A9, QEMU vexpress-a9

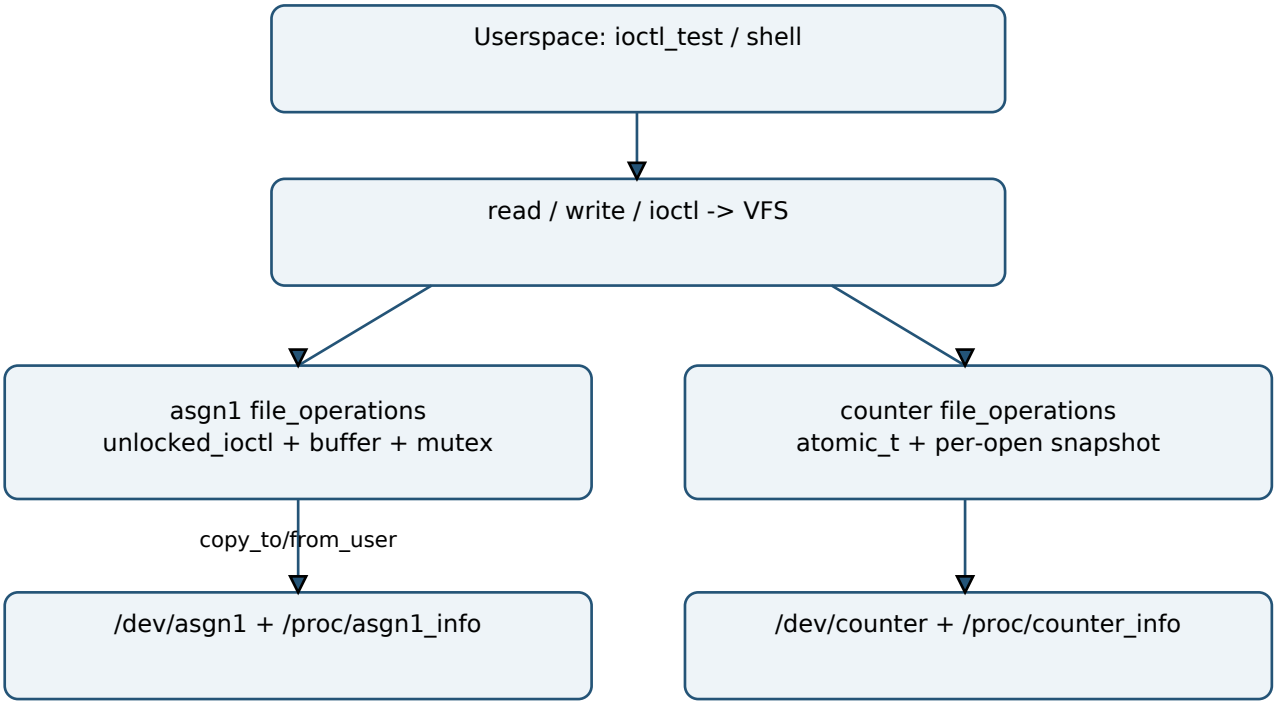
This report examines two implemented interfaces: a fixed-major character driver with ioctl commands, and a misc counter device. The design emphasizes explicit ABI boundaries, coherent statistics, bounded user copies, partial-read behavior, and cleanup on every initialization failure.

Scope of evidence: both modules were cross-compiled and executed inside ARM QEMU. The results are software emulation results, not measurements on physical hardware. Source files, executable tests, and complete console/dmesg logs accompany this report.

The ioctl driver is named asgn1_driver and creates /dev/asgn1 at major 241. The independent misc driver creates /dev/counter at major 10 with a dynamic minor. Existing LAB02 device names and binaries are not replaced by these assignment devices.

Reading guide: architecture; ABI and mode design; synchronization and resource ownership; counter semantics; driver comparison; test evidence; theory; reproduction and references.

1. Architecture and interface boundaries



Applications enter through read(), write(), or ioctl(). VFS resolves the opened character device to its file_operations table. The ioctl callback validates the complete encoded request before copying arguments. GET_STATS copies a local, coherent snapshot back to userspace; SET_MODE copies an integer into kernel memory before updating shared state.

The diagram includes both drivers because their userspace-facing system calls are similar even though registration differs. The proc files use seq_file callbacks rather than the character-device read callback. This separation lets diagnostics inspect state without consuming device data or incrementing the device open/read counters.

Data-plane traffic uses read/write; mode changes, reset, version, and structured statistics use the control plane. No userspace pointer is retained after a syscall returns. Internal memcpy is used only between valid kernel buffers; user transfers use uaccess helpers.

2. ioctl ABI and state transitions

Command	Direction / size	Behavior
RESET_BUFFER	_IO / 0	Clear all buffer bytes and read/write counters
GET_STATS	_IOR / 20	Return five 32-bit integer fields
SET_MODE	_IOW / 4	0: prefix [KERNEL]; 1: raw echo
GET_VERSION	_IOR / 64	Return zero-filled bounded version string

The ABI header is shared by the module and the test executable. It uses magic T, request numbers 0 through 3, and fixed-size int fields on the tested ARM EABI and x86_64 ABI. No pointers, long values, or compiler-dependent enums occur in the exported stats structure.

For GET_STATS, the captured ARM command is 0x80145401: _IOC_NR gives 1, _IOC_TYPE gives 0x54, and _IOC_SIZE gives 20. The high direction bits encode a transfer to the caller. Direction names are from the userspace point of view. Matching only the request number would accidentally accept malformed commands with incompatible sizes; the switch matches the full command value.

Echo mode is the load-time default. Kernel mode prefixes exactly eight bytes, [KERNEL], without adding a space. last_write_size records accepted payload bytes, whereas buffer_len includes the prefix. RESET preserves open_count and the selected mode, but resets read_count, write_count, buffer_len and last_write_size.

Unknown commands return ENOTTY, invalid mode values return EINVAL, and bad argument pointers return EFAULT. GET_VERSION uses a 64-byte initialized local array, avoiding leakage of uninitialized stack bytes.

3. Buffer safety, counters and lifetime

The internal buffer capacity is 1024 bytes, including the trailing NUL. A write accepts at most capacity minus one minus the active prefix length. It first copies payload to a temporary kernel buffer, then commits the fully formed string and metadata while holding data_lock. A failed copy_from_user cannot partially overwrite previously stored data.

The same mutex covers reads, writes, resets, mode changes and statistics snapshots. read_count counts successful read calls returning positive data, rather than open calls or EOF probes. write_count counts successful nonempty writes. Zero-length writes return zero without changing state. proc output holds the mutex while formatting related fields.

Mutexes are suitable because user copies can fault and sleep. A spinlock must not surround such operations. GET_STATS releases the mutex after creating the local snapshot, before copying it out: the result is coherent at snapshot time, although another writer can change the live device immediately afterward.

Initialization acquires resources in order: character major, class, device, proc entry. Each failure label reverses exactly the successful earlier steps. Exit removes proc first, then the device, class and character registration. file_operations.owner is THIS_MODULE, so an open device pins the module.

No asynchronous worker or timer is needed. Writes temporarily allocate a bounded kernel payload buffer, then free it before returning. This avoids a 1 KiB stack frame warning on ARM. Each successful syscall completes all its accesses; no user buffer address escapes into shared state.

4. Misc counter: atomic state and EOF

The counter driver uses misc_register with MISC_DYNAMIC_MINOR. The misc subsystem supplies major 10 and chooses a free minor. Device creation is delegated to the device model/devtmpfs. It does not reserve another experimental major using register_chrdev.

The value uses atomic_t and atomic_read, atomic_set and atomic_inc as required. A mutex additionally serializes the compound operation of taking the current value, incrementing it, and updating total_reads. An atomic increment alone would not make an independently loaded return value and the statistics snapshot mutually consistent.

Each open file description receives a small kzalloc-owned reader_state. Its first nonempty read creates one decimal snapshot and increments the global counter once. Partial reads consume that same snapshot using simple_read_from_buffer and the file offset. Once all bytes are consumed, the callback returns zero. The release callback frees the per-open state.

This snapshot policy prevents a one-byte reader from combining digits from different counter values. The test uses dd with bs=1 to verify the full string 123 and EOF. Separate cat invocations create separate opens and therefore consume successive values.

Writes accept bounded ASCII strings, reject embedded NUL bytes, call kstrtoint, reject negative values, and update total_writes only after successful validation. The extra overflow policy wraps INT_MAX to zero after returning INT_MAX once, preserving nonnegative output. That boundary policy is explicit rather than relying on signed overflow.

5. Character, block and network devices

Criterion	Character	Block	Network
Operation interface	file_operations	block_device_operations + blk-mq	net_device_ops
Transfer unit	Bytes / records	Sectors and requests	Packets / frames
Buffering	Driver-defined; no universal cache	Page cache and block queues; direct I/O can bypass cache	skb queues, socket buffers, NAPI
Embedded example	UART, sensor control	eMMC, SD, NVMe	Ethernet, Wi-Fi
Typical access	open/read/write/ioctl	Filesystem or raw block I/O	Sockets, network stack

A sensor with short control records is a character-device candidate: it needs read/write and a few configuration requests, not filesystem sectors. A bulk data logger backed by eMMC should use the existing block device and filesystem; introducing a custom byte-stream driver would lose standard caching and filesystem integration.

A packet interface such as an Ethernet controller belongs in the network subsystem. Exposing frames only through a custom character device would bypass routing, sockets, protocol handling, and the usual network management tools. A custom serial protocol on a UART may still remain a character interface because the hardware abstraction is a serial byte stream.

For a small single-purpose counter, miscdevice reduces registration boilerplate. A fixed major character driver is useful here to study the explicit registration path, but fixed numbers require coordination and can collide. Neither framework changes the need for safe callbacks, lifetime management and synchronization.

6. ioctl verification on ARM

The userspace tester opens the device, verifies the default echo mode, selects kernel mode, checks all five statistics fields, reads the version, tests invalid mode/command/pointer, and resets the device. The excerpt below is the real terminal capture; repeated dmesg playback is retained in the separate full log.

```
/opt/asgn1/task1_ioctl/test_asgn1.sh; echo EXIT=$?
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: loaded major=241 mode=echo
read verified: [echo]
asgn1: ioctl cmd=0x40045402 nr=2 size=4
read verified: [[KERNEL]test]
asgn1: ioctl cmd=0x80145401 nr=1 size=20
open_count=3 write_count=2 read_count=2 buffer_len=12 last_write_size=4
asgn1: ioctl cmd=0x80405403 nr=3 size=64
version=asgn1_driver v1.0 - SE205068; stats_size=20 cmd=0x80145401
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x5463 nr=99 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x5400 nr=0 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
read verified: []
asgn1: ioctl cmd=0x40045402 nr=2 size=4
IOCTL TEST PASSED
open_count: 4
write_count: 0
read_count: 0
buffer_len: 0
last_write_size: 0
mode: 1
asgn1: unloaded
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: loaded major=241 mode=echo
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x80405403 nr=3 size=64
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x5463 nr=99 size=0
```

The first GET_STATS result is asserted by the executable, not merely printed: open_count=3, write_count=2, read_count=2, buffer_len=12 and last_write_size=4. After reset, both read/write counters and buffer length are asserted zero; a fresh read must return EOF immediately.

7. Counter verification and debugging

```
/opt/asgn1/task2_misc/test_counter.sh; echo EXIT=$?
counter: loaded major=10 minor=125
counter: 0
total_reads: 0
total_writes: 0
random: fast init done
counter: 100
total_reads: 2
total_writes: 1
counter: 0
total_reads: 3
total_writes: 2
sh: write error: Invalid argument
sh: write error: Invalid argument
counter: unloaded
hw perfevents: enabled with armv7_cortex_a9 PMU driver, 7 counters available
counter: loaded major=10 minor=125
counter: unloaded
TASK2 PASSED
EXIT=0
```

The three proc snapshots show the required phases: before writes, after setting 100, and after resetting zero. Invalid negative and nonnumeric writes return EINVAL. The script also verifies two successive reads yield 0 then 1, and a one-byte reader terminates without an infinite loop.

A real build problem occurred while preparing the target kernel: the assembler rejected dmb ish with the selected processor mode. The configuration selected ARMv7, but the toolchain invocation needed an explicit architecture flag. Rebuilding with KCFLAGS=-march=armv7-a produced the Cortex-A9 image and enabled the subsequent module builds. The build command is preserved in the support scripts.

The test logs show clean module unload messages. The first external module emits the normal out-of-tree taint notice; that notice is not an Oops or failed insertion. The kernel also prints unrelated emulated display/flash messages during boot and poweroff. Driver validation distinguishes those platform messages from module failures.

8. Theory: module lifetime and user access

Reference counting: file_operations.owner identifies the owning module. VFS obtains a module reference while a file using these operations remains open and drops it when the final reference to the file is released. Explicit try_module_get/module_put can manage additional references when a subsystem needs them. Initialization references and all callback lifetimes must also be respected.

A normal rmmod of a driver with a held-open device fails because the module is in use. The safe procedure is to stop users, close every inherited or duplicated descriptor, and retry. Forced removal uses rmmod -f only if CONFIG_MODULE_FORCE_UNLOAD permits it; it can leave executable callbacks pointing into freed code and cause corruption or a kernel crash. The assignment tests do not force unload.

User copies: a userspace virtual address belongs to the current process address space, may be unmapped, read-only, or paged out, and can change during the operation. copy_to_user checks access constraints and uses exception-table handling so faults become a failed copy rather than an unhandled kernel exception.

The MMU does not translate a pointer according to the C variable type. It applies page tables and privilege/access permissions. On ARM, protections such as PAN or software equivalents can restrict kernel access to user mappings. Direct dereference may fault or violate the uaccess contract even if an address appeared valid earlier. memcpy alone supplies neither the required validation nor recoverable fault behavior. Never treat user pointers as trusted kernel pointers.

9. Theory, reproduction and references

MTD models raw flash erase/read/write geometry, eraseblocks, bad blocks and out-of-band data. Raw NAND and NOR can require MTD-aware filesystems such as UBIFS or JFFS2. eMMC and managed SD expose logical sectors behind an internal flash controller and normally belong to the block layer; their internal physical NAND does not mean applications should access them as raw MTD.

EOF handling: cat repeatedly calls read until it receives zero, an error, or is interrupted. The kernel dispatches each userspace read through VFS into the driver; VFS does not invent an EOF merely because the callback previously returned one number. Without offset/snapshot handling, an always-positive driver would feed cat forever and continuously increment the counter.

```
bash assignment_support/scripts/run-qemu.sh 1
# In the guest:
/opt/asgn1/task1_ioctl/test_asgn1.sh
/opt/asgn1/task2_misc/test_counter.sh
```

Rebuild with the included kernel configuration and ARM GNU toolchain, then set KDIR and run assignment_support/scripts/build.sh. make and make clean are supported in both module directories. The kernel archive/build tree is intentionally excluded from Git; the resulting boot image, DTB, configuration and modules are included.

References: Linux 5.15 source, include/linux/fs.h, miscdevice.h, atomic APIs, lib/kstrtox.c and libfs.c; official ioctl guide: https://docs.kernel.org/driver-api/ioctl.html ; ioctl encoding: https://kernel.org/doc/html/v6.9/userspace-api/ioctl/ioctl-number.html ; Linux read/ioctl manuals: https://man7.org/linux/man-pages/man2/read.2.html and https://man7.org/linux/man-pages/man2/ioctl.2.html . References informed API usage; test outputs are from the included implementation.

Appendix - Complete ioctl test output

```
/opt/asgn1/task1_ioctl/test_asgn1.sh; echo EXIT=$?
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: loaded major=241 mode=echo
read verified: [echo]
asgn1: ioctl cmd=0x40045402 nr=2 size=4
read verified: [[KERNEL]test]
asgn1: ioctl cmd=0x80145401 nr=1 size=20
open_count=3 write_count=2 read_count=2 buffer_len=12 last_write_size=4
asgn1: ioctl cmd=0x80405403 nr=3 size=64
version=asgn1_driver v1.0 - SE205068; stats_size=20 cmd=0x80145401
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x5463 nr=99 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x5400 nr=0 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
read verified: []
asgn1: ioctl cmd=0x40045402 nr=2 size=4
IOCTL TEST PASSED
open_count: 4
write_count: 0
read_count: 0
buffer_len: 0
last_write_size: 0
mode: 1
asgn1: unloaded
asgn1_driver: loading out-of-tree module taints kernel.
asgn1: loaded major=241 mode=echo
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x80405403 nr=3 size=64
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: ioctl cmd=0x5463 nr=99 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x5400 nr=0 size=0
asgn1: ioctl cmd=0x80145401 nr=1 size=20
asgn1: ioctl cmd=0x40045402 nr=2 size=4
asgn1: unloaded
TASK1 PASSED
EXIT=0
```